API Test Report — AI Platform Integration Testing

Overview

This report documents API testing conducted against a representative AI chat completion API: Hugging Face's Inference Router, using Llama 3.1 8B as the underlying model. The testing approach mirrors what a Solution Consultant would run before certifying a client integration as ready for go-live with an AI platform like Travtus Adam.

The purpose was to validate API behaviour across authentication, error handling, data validation, and boundary conditions. These checks are designed to catch the failure modes most likely to surface during a live client deployment.

API tested: Hugging Face Inference Router (chat completions and model metadata endpoints)
Tool used: Postman
Total requests built: 8
Date: [today's date]

Test Results

#	Test Name	Expected Status	Actual Status	Pass/Fail
1	Valid chat message	200	200	Pass
2	Empty message body	200 or 400	200	Pass
3	Invalid API key	401	401	Pass
4	No API key	401 or 403	401	Pass
5	Long message boundary	200	200 (3.43s)	Pass
6	Wrong data type	400 or 422	400	Pass
7	Model info (valid key)	200	200	Pass

#	Test Name	Expected Status	Actual Status	Pass/Fail
8	Invalid model endpoint	400 or 404	400	Pass

All 8 requests passed their assertions. 16 individual test cases ran across the suite, all green.

Key Findings

Authentication testing. The API consistently returns 401 for both missing and invalid keys, which is the correct behaviour. If a client integration loses its API key mid-session, it will fail loudly with a clear error rather than silently passing bad data downstream. For a Solution Consultant onboarding a new client, this means the auth layer should surface key rotation issues immediately rather than letting them quietly corrupt resident communications.

Error handling. The data type validation test returned a clear, structured error explaining exactly what was wrong: "body unmarshal json: cannot unmarshal number into Go struct field." Client-side code can parse this and present it in a useful way to property managers. The empty message test returned 200 with the AI generating a generic response, which means an integration needs to apply its own input validation rather than relying on the API to reject empty inputs.

Boundary behaviour. The long message test (roughly 850 characters of resident input) completed successfully in 3.43 seconds. This is well within an acceptable response time for resident communications. For Travtus Adam, this suggests that even verbose maintenance complaints or detailed lease queries can be processed without timeout risk. However, a three-second response window means the client UI should show a "thinking" indicator so users do not assume the system has stalled.

Pre-Deployment Recommendations

Before any client goes live with an AI communication platform like Adam, the following API checks should be run as part of the standard onboarding validation:

1. Confirm API keys are stored in environment variables, not hardcoded in any client config file or version control.

2. Set up monitoring alerts for any spike in 401 responses, which signal key expiry or misconfiguration.
3. Test the exact API call patterns the client's volume will generate, including burst conditions during high-traffic periods like rent due dates or building emergencies.
4. Validate error response formats so client-side code can handle failures gracefully and surface meaningful messages to property managers.
5. Document expected response times under realistic message-length distributions, so escalation rules can be calibrated to actual performance rather than assumed performance.
6. Test with deliberately malformed inputs (wrong data types, oversized payloads, special characters) to confirm the integration layer's input validation catches issues before they reach the AI.

The tests in this suite represent a baseline for any AI platform deployment. Specific client integrations would extend this with tests against the client's own systems (Yardi, Zendesk, Salesforce, SendGrid) using the same methodology.